

GhostIntel — Technical Specification

Target: GhostIntel 3.x

Architecture: Modular Core + CLI-first + Web-ready

Database: SQLite

Primary language: Python

Supported: Linux / Windows / Termux

Localization: ID / EN

Official Website: <https://www.ghintel.web.id>

1. Repository Architecture

```
GhostIntel/
|
├─ ghostintel.py
├─ pyproject.toml
├─ requirements.txt
├─ requirements-dev.txt
├─ README.md
├─ CHANGELOG.md
├─ CONTRIBUTING.md
├─ SECURITY.md
├─ LICENSE
├─ .gitignore
|
├─ core/
|   └─ __init__.py
|   └─ engine.py
|   └─ config.py
|   └─ database.py
|   └─ request_manager.py
|   └─ api_manager.py
|   └─ rate_limiter.py
|   └─ cache.py
|   └─ proxy.py
|   └─ logger.py
|   └─ debug.py
|   └─ bug_reporter.py
|   └─ i18n.py
|   └─ module_registry.py
|   └─ findings.py
|   └─ correlation.py
|   └─ monitor.py
|   └─ notifier.py
|   └─ updater.py
|   └─ doctor.py
|   └─ exceptions.py
|   └─ models.py
|
├─ modules/
|   └─ __init__.py
|   └─ username.py
|   └─ email.py
|   └─ phone.py
|   └─ domain.py
|   └─ subdomain.py
|   └─ dns.py
```

```
|   ├── ip.py
|   ├── asn.py
|   ├── certificate.py
|   ├── breach.py
|   ├── reputation.py
|   └── dorking.py
|
└── sources/
    ├── __init__.py
    ├── username/
    ├── email/
    ├── phone/
    ├── domain/
    └── common/
|
└── dorks/
    ├── level1.yaml
    └── level2.yaml
|
└── locales/
    ├── id.json
    └── en.json
|
└── reports/
    ├── __init__.py
    ├── generator.py
    ├── schema.py
    └── templates/
        ├── html/
        ├── markdown/
        └── txt/
|
└── notifications/
    ├── __init__.py
    └── telegram.py
|
└── web/
    ├── __init__.py
    ├── app.py
    ├── routes/
    ├── templates/
    └── static/
|
└── tests/
    ├── unit/
    ├── integration/
    ├── e2e/
    └── fixtures/
|
└── examples/
    └── config.example.yaml
|
└── docs/
    ├── installation.md
    ├── termux.md
    ├── configuration.md
    ├── modules.md
    ├── username.md
    ├── email.md
    ├── phone.md
    ├── domain.md
    └── subdomain.md
```

```
|— dorking.md
|— api.md
|— telegram.md
|— reports.md
|— monitoring.md
|— troubleshooting.md
|— development.md
```

2. Root Files

ghostintel.py

Entry point.

Tugas:

```
CLI startup
↓
Load configuration
↓
Initialize database
↓
Initialize i18n
↓
Initialize logging
↓
Register modules
↓
Parse command
↓
Execute command
```

Jangan memasukkan logic username/domain/API di sini.

File ini hanya menjadi **bootstrap + CLI entry point**.

3. pyproject.toml

Menentukan:

```
package metadata
version
dependencies
CLI entrypoint
Python version
build configuration
```

CLI idealnya bisa dipanggil:

```
ghostintel
```

setelah installation.

4. requirements.txt

Production dependencies.

Contoh kategori:

```
CLI framework
HTTP client
DNS library
Phone parser
YAML parser
Database helper
HTML/report utilities
```

Dependency harus seminimal mungkin.

5. requirements-dev.txt

Development-only:

```
pytest
pytest-cov
mocking tools
linting
formatting
type checking
```

Production installation tidak perlu membawa seluruh development stack.

6. core/

Ini adalah **jantung GhostIntel**.

Aturan penting:

Module tidak boleh mengimplementasikan ulang fungsi core yang sudah tersedia.

Contoh:

✗ `username.py` membuat HTTP client sendiri.

✗ `email.py` membuat rate limiter sendiri.

✗ `telegram.py` menyimpan token sendiri.

Semua lewat core.

7. core/engine.py

Central orchestration.

Tanggung jawab:

```
run_module()
run_scan()
run_multiple_modules()
collect_results()
handle_module_error()
save_results()
generate_report()
trigger_notifications()
```

Flow:

```
CLI
↓
Engine
↓
Module
↓
Finding
↓
Database
↓
Report
↓
Notification
```

Engine tidak mengetahui detail platform tertentu.

8. `core/config.py`

Configuration manager.

Fungsi:

```
load()
save()
get()
set()
reset()
validate()
```

Config source priority:

```
CLI arguments
↓
Environment
↓
Config file
↓
Default
```

9. `core/database.py`

Database abstraction layer.

Jangan module langsung melakukan SQL random.

Interface:

```
database.save_scan(...)
database.get_scan(...)
database.save_finding(...)
database.get_findings(...)
database.save_api_provider(...)
```

Tanggung jawab:

```
connection
migration
transaction
CRUD
backup
integrity check
```

10. `core/request_manager.py`

Semua network request melalui file ini.

Interface konseptual:

```
request_manager.get(...)
request_manager.post(...)
request_manager.request(...)
```

Menangani:

```
timeout
retry
headers
proxy
rate limit
cache
HTTP errors
connection errors
logging
```

Response harus dinormalisasi.

11. `core/api_manager.py`

API abstraction.

Contoh:

```
api_manager.get_provider("hibp")
api_manager.test_provider("hibp")
api_manager.call("hibp", ...)
```

Provider tidak boleh membaca API key dari environment/config secara langsung.

API Manager yang menyediakan credential.

12. `core/rate_limiter.py`

Global + provider-specific.

Model:

```
RateLimiter
├─ global
├─ provider
├─ module
└─ endpoint
```

Contoh:

```
Telegram → 30/min
Provider A → 100/min
Provider B → 10/min
```

Nilai aktual harus configurable dan mengikuti batas provider.

13. `core/cache.py`

Caching.

Key:

```
module
target
provider
parameters
```

TTL:

```
default
provider-specific
user-configured
```

Cache tidak boleh membuat data lama terlihat sebagai data real-time tanpa timestamp.

14. `core/proxy.py`

Mengelola proxy.

```
proxy.get()
proxy.validate()
proxy.enabled()
```

Request Manager yang menggunakannya.

Module tidak perlu tahu detail proxy.

15. `core/logger.py`

Central logging.

Levels:

```
DEBUG
INFO
WARNING
ERROR
CRITICAL
```

Output:

```
terminal
file
```

Sensitive values otomatis diremask.

16. `core/debug.py`

Debug mode.

Fungsi:

```
enable_debug()
create_debug_bundle()
capture_exception()
sanitize()
get_environment()
```

Debug bundle:

```
version
OS
Python
module
command
traceback
sanitized config
logs
```

17. `core/bug_reporter.py`

Bug report builder.

Input:

```
bug description
expected
actual
module
version
bug ID
```

Output:

```
structured bug report
```

Tidak otomatis mengirim target sensitif.

18. `core/i18n.py`

Central translation system.

API:

```
t("scan.start")
t("result.found")
t("help.website")
```

Language:

```
id
en
```

Fallback:

```
requested language
↓
English
↓
key
```

Jika translation missing, jangan crash.

19. `core/module_registry.py`

Registry seluruh module.

```
register(module)
get(name)
list()
enable(name)
disable(name)
```

Setiap module memiliki metadata:

```
name
version
description
category
dependencies
capabilities
```

20. `core/findings.py`

Standard result object.

Minimal:

```
Finding(  
    target=...,  
    module=...,  
    source=...,  
    status=...,  
    confidence=...,  
    evidence=...,  
    metadata=...,  
    timestamp=...  
)
```

21. Status Enum

```
FOUND  
NOT_FOUND  
NOT_ACTIVE  
UNKNOWN  
ERROR  
TIMEOUT  
RATE_LIMITED  
BLOCKED
```

Status wajib terkontrol melalui enum.

Jangan string random:

```
"found"  
"FOUND!"  
"yes"  
"exist"
```

22. Confidence Enum

```
HIGH  
MEDIUM  
LOW  
UNKNOWN
```

Confidence bukan pengganti status.

Contoh valid:

```
status=FOUND  
confidence=HIGH
```

23. `core/correlation.py`

Menghubungkan findings.

Input:

```
Finding[]
```

Output:

```
Correlation[]
```

Correlation harus menyimpan:

```
source finding
related finding
relationship
confidence
evidence
```

24. `core/monitor.py`

Monitoring engine.

Tugas:

```
create target
schedule scan
store snapshot
compare snapshot
detect changes
generate event
```

25. `core/notifier.py`

Notification abstraction.

```
notifier.send(event)
```

Telegram hanya salah satu implementation.

Architecture:

```
Notifier
├── Telegram
└── Future provider
```

26. `core/updater.py`

Tugas:

```
check_version()
get_manifest()
compare_versions()
download()
verify_checksum()
backup()
install()
rollback()
```

Updater tidak boleh menyentuh:

```
~/ghostintel/ghostintel.db
```

secara destructive.

27. `core/doctor.py`

System diagnostic.

Command:

```
ghostintel doctor
```

Checks:

```
Python
dependencies
database
config
filesystem
network
API
Telegram
locale
modules
updater
```

28. `core/exceptions.py`

Custom exception hierarchy.

Contoh:

```
GhostIntelError
├─ ConfigurationError
├─ DatabaseError
├─ NetworkError
├─ APIError
├─ RateLimitError
├─ ModuleError
├─ ReportError
├─ UpdateError
└─ ValidationError
```

29. `core/models.py`

Shared models.

Contoh:

```
Target
Scan
Finding
Source
APIProvider
NotificationEvent
MonitorTarget
```

Jangan membuat model duplicate di tiap module.

30. `modules/`

Berisi business logic OSINT.

Setiap module harus menggunakan:

```
RequestManager
APIManager
Database
Findings
Logger
i18n
```

sesuai kebutuhan.

31. Username Module

File:

```
modules/username.py
```

Tugas:

```
validate username
load platform adapters
execute checks
validate response
extract metadata
create findings
```

Jangan menganggap:

```
HTTP 200 = FOUND
```

32. Username Sources

Struktur:

```
sources/username/  
├─ github.py  
├─ instagram.py  
├─ reddit.py  
├─ tiktok.py  
├─ x.py  
└─ ...
```

Ini lebih scalable daripada membuat semua platform menjadi satu file 5.000 baris.

33. Platform Adapter Contract

Setiap platform:

```
class PlatformAdapter:  
    name  
    url_template  
  
    def check(username):  
        ...  
  
    def validate(response):  
        ...  
  
    def extract(response):  
        ...
```

34. Platform Result

Konsep:

```
PlatformResult(  
    platform="example",  
    username="target",  
    status=FOUND,  
    profile_url="...",  
    metadata={},  
    evidence=[],  
)
```

Metadata optional.

35. Username Metadata

Possible fields:

```
display_name
bio
avatar
followers
following
posts
videos
account_created
verified
website
location
```

Jika unavailable:

```
NULL
```

Bukan:

```
0
false
unknown
```

kecuali memang nilai tersebut benar-benar diketahui.

36. Email Module

```
modules/email.py
sources/email/
```

Basic checks:

```
syntax
domain
MX
SPF
DMARC
disposable
role
```

Advanced:

```
API enrichment
breach
reputation
```

37. Phone Module

```
modules/phone.py
```

Fields:

```
normalized number
country
region
carrier
line type
timezone
validity
```

38. Domain Module

```
modules/domain.py
```

Orchestrates:

```
DNS
WHOIS
HTTP
HTTPS
TLS
technology
IP
ASN
subdomain
```

Domain module tidak perlu mengimplementasikan semuanya sendiri.

Ia memanggil module/service lain.

39. DNS Module

```
modules/dns.py
```

Records:

```
A
AAAA
MX
NS
TXT
CNAME
CAA
SOA
SRV
```

40. Subdomain Module

```
modules/subdomain.py
```

Output standard:

```
hostname
source
resolved_ip
http_status
https
technology
```

Architecture harus memungkinkan future integration dengan `sub-intel`.

41. IP Module

```
modules/ip.py
```

Fields:

```
ip
version
country
region
city
asn
isp
organization
reverse_dns
hostname
reputation
```

42. ASN Module

```
modules/asn.py
```

Fields:

```
asn
organization
country
prefixes
```

43. Certificate Module

```
modules/certificate.py
```

Fields:

```
issuer
subject
SAN
valid_from
valid_until
serial
fingerprint
```

44. Breach Module

```
modules/breach.py
```

Harus provider-based.

```
BreachProvider
├── Provider A
├── Provider B
└── ...
```

Jika provider tidak tersedia:

```
UNKNOWN
```

bukan:

```
NOT_FOUND
```

45. Reputation Module

Sama:

```
ReputationProvider
```

Setiap provider menghasilkan normalized Finding.

46. Dorking Module

```
modules/dorking.py
```

Tugas:

```
load templates
validate target
render templates
categorize
generate search URLs
export
```

Template:

```
dorks/
├── level1.yaml
└── level2.yaml
```

47. Dork Template Schema

Contoh:

```
- id: site_basic
  level: 1
  title: Site Search
  query: "site:{domain}"
  description: "Find indexed pages for a domain"
```

Level 2:

```
- id: sensitive_documents
  level: 2
  title: Public Documents
  query: "site:{domain} (filetype:pdf OR filetype:doc OR filetype:docx)"
  description: "Find publicly indexed documents"
```

48. Reports

reports/schema.py

Mendefinisikan:

```
Report
ReportSummary
ReportFinding
ReportSource
```

49. Report Generator

reports/generator.py

API:

```
generate_json(report)
generate_html(report)
generate_markdown(report)
generate_txt(report)
```

Semua output menggunakan data yang sama.

50. Templates

```
reports/templates/
├─ html/
├─ markdown/
└─ txt/
```

Template tidak boleh mengandung business logic.

51. Telegram

notifications/telegram.py

Tanggung jawab:

```
authenticate bot
send message
queue
batch
deduplicate
rate limit
retry
```

Jangan mengirim setiap finding secara individual.

52. Web

Web layer hanya menjadi presentation/API layer.

```
web/
├─ app.py
├─ routes/
├─ templates/
└─ static/
```

Jangan menaruh logic OSINT langsung di route.

Route:

```
HTTP
↓
Engine
↓
Module
```

53. Database Schema

Sekarang bagian pentingnya.

settings

```
CREATE TABLE settings (
  key TEXT PRIMARY KEY,
  value TEXT,
  updated_at TEXT NOT NULL
);
```

Contoh:

```
language = id
theme = dark
default_report = html
```

54. **api_providers**

```
CREATE TABLE api_providers (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL UNIQUE,  
  enabled INTEGER NOT NULL DEFAULT 0,  
  status TEXT,  
  last_tested_at TEXT,  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);
```

API secret jangan disimpan plaintext kalau secure storage tersedia.

55. targets

```
CREATE TABLE targets (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  type TEXT NOT NULL,  
  value TEXT NOT NULL,  
  normalized_value TEXT NOT NULL,  
  created_at TEXT NOT NULL  
);
```

Unique target bisa memakai:

```
type + normalized_value
```

56. scans

```
CREATE TABLE scans (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  target_id INTEGER,  
  scan_type TEXT NOT NULL,  
  status TEXT NOT NULL,  
  started_at TEXT NOT NULL,  
  completed_at TEXT,  
  duration_ms INTEGER,  
  error TEXT,  
  FOREIGN KEY(target_id) REFERENCES targets(id)  
);
```

57. findings

```
CREATE TABLE findings (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  scan_id INTEGER NOT NULL,  
  module TEXT NOT NULL,  
  source TEXT,  
  status TEXT NOT NULL,  
  confidence TEXT,  
  target TEXT,  
  evidence TEXT,  
  metadata TEXT,  
  created_at TEXT NOT NULL,  
  FOREIGN KEY(scan_id) REFERENCES scans(id)  
);
```

metadata dapat disimpan sebagai JSON.

58. sources

```
CREATE TABLE sources (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  finding_id INTEGER NOT NULL,  
  name TEXT,  
  url TEXT,  
  evidence TEXT,  
  checked_at TEXT,  
  FOREIGN KEY(finding_id) REFERENCES findings(id)  
);
```

59. monitor_targets

```
CREATE TABLE monitor_targets (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  target_id INTEGER NOT NULL,  
  interval_seconds INTEGER NOT NULL,  
  enabled INTEGER NOT NULL DEFAULT 1,  
  last_run_at TEXT,  
  next_run_at TEXT,  
  created_at TEXT NOT NULL,  
  FOREIGN KEY(target_id) REFERENCES targets(id)  
);
```

60. monitor_snapshots

```
CREATE TABLE monitor_snapshots (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  monitor_target_id INTEGER NOT NULL,  
  snapshot_hash TEXT NOT NULL,  
  data TEXT NOT NULL,  
  created_at TEXT NOT NULL,  
  FOREIGN KEY(monitor_target_id) REFERENCES monitor_targets(id)  
);
```

61. notifications

```
CREATE TABLE notifications (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  event_type TEXT NOT NULL,  
  channel TEXT NOT NULL,  
  status TEXT NOT NULL,  
  payload TEXT,  
  attempts INTEGER DEFAULT 0,  
  last_attempt_at TEXT,  
  sent_at TEXT,  
  created_at TEXT NOT NULL  
);
```

62. bug_reports

```
CREATE TABLE bug_reports (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  bug_id TEXT NOT NULL UNIQUE,  
  module TEXT,  
  title TEXT,  
  description TEXT,  
  expected TEXT,  
  actual TEXT,  
  debug_bundle TEXT,  
  status TEXT DEFAULT 'OPEN',  
  created_at TEXT NOT NULL  
);
```

63. update_cache

```
CREATE TABLE update_cache (  
  id INTEGER PRIMARY KEY CHECK (id = 1),  
  current_version TEXT NOT NULL,  
  latest_version TEXT,  
  checked_at TEXT,  
  manifest TEXT  
);
```

64. Database Relationship

```
targets
├── scans
│   ├── findings
│   │   └── sources
```

```
targets
├── monitor_targets
│   └── monitor_snapshots
```

65. CLI Command Tree

Sekarang kita lock CLI.

```
ghostintel
├── username
│   ├── <username>
│   └── --help
├── email
│   ├── <email>
│   └── --help
├── phone
│   ├── <number>
│   └── --help
├── domain
│   ├── <domain>
│   └── --help
├── subdomain
│   ├── <domain>
│   └── --help
├── dns
│   ├── <domain>
│   └── --help
├── ip
│   ├── <ip>
│   └── --help
├── asn
│   ├── <asn>
│   └── --help
├── certificate
│   ├── <domain>
│   └── --help
├── breach
│   └── <target>
```

```
|   └─ --help
|
└─ reputation
|   └─ <target>
|   └─ --help
|
└─ dork
|   └─ <domain>
|   └─ --help
|
└─ scan
|   └─ ...
|   └─ --help
|
└─ report
|   └─ list
|   └─ view
|   └─ export
|   └─ delete
|
└─ monitor
|   └─ add
|   └─ list
|   └─ remove
|   └─ run
|   └─ status
|
└─ api
|   └─ list
|   └─ set
|   └─ remove
|   └─ test
|   └─ status
|
└─ config
|   └─ get
|   └─ set
|   └─ list
|   └─ reset
|
└─ telegram
|   └─ setup
|   └─ test
|   └─ status
|
└─ bug
|   └─ create
|   └─ list
|   └─ view
|
└─ debug
|   └─ logs
|   └─ bundle
|   └─ clear
|
└─ doctor
|
└─ update
|   └─ check
|   └─ install
|
└─ version
```

```
|  
└─ help
```

66. Username CLI Contract

```
ghostintel username <username>
```

Options:

```
--deep  
--save  
--format  
--output  
--no-cache  
--timeout  
--proxy  
--telegram  
--json
```

Contoh:

```
ghostintel username example --deep --save --format html
```

67. Email CLI

```
ghostintel email user@example.com
```

Options:

```
--deep  
--breach  
--reputation  
--save  
--format
```

68. Domain CLI

```
ghostintel domain example.com
```

Options:

```
--deep  
--dns  
--subdomains  
--certificate  
--technology  
--save
```

69. Dork CLI

```
ghostintel dork example.com
```

Options:

```
--level 1
--level 2
--category
--engine google
--engine bing
--engine duckduckgo
--save
```

Output:

```
Generated Dorks:
01 ...
02 ...
03 ...
```

70. Scan CLI

Combined:

```
ghostintel scan <target>
```

atau explicit:

```
ghostintel scan \
  --username example \
  --email user@example.com \
  --domain example.com
```

Options:

```
--modules
--deep
--save
--format
--telegram
--no-cache
```

71. API CLI

```
ghostintel api list
```

Output:

Provider	Status
Provider A	Connected
Provider B	Not Configured
Provider C	Invalid

Test:

```
ghostintel api test provider-a
```

72. Config CLI

```
ghostintel config get language
```

```
ghostintel config set language en
```

```
ghostintel config list
```

73. Telegram CLI

```
ghostintel telegram setup
```

```
ghostintel telegram test
```

```
ghostintel telegram status
```

74. Report CLI

```
ghostintel report list
```

```
ghostintel report view 123
```

```
ghostintel report export 123 --format html
```

75. Monitor CLI

```
ghostintel monitor add example.com
```

```
ghostintel monitor list
```

```
ghostintel monitor status
```

```
ghostintel monitor run 1
```

76. Update CLI

```
ghostintel update check
```

Jika ada:

```
GhostIntel 3.0.0
Latest      3.5.0

Update available.
```

Install:

```
ghostintel update install
```

77. Doctor CLI

```
ghostintel doctor
```

Contoh:

```
GhostIntel Doctor

Core           OK
Database       OK
Configuration   OK
Network        OK
Modules        12/12 OK
API            3/4 OK
Telegram       OK
Reports        OK
Updater        OK
Locale         OK

System is ready.
```

78. Module Contract

Ini bagian yang paling penting untuk menjaga codebase tetap bersih.

Semua module harus mengikuti contract.

Konsep:

```
class BaseModule:
    name: str
    version: str
    description: str
    category: str

    def validate_target(self, target):
        ...

    def scan(self, target, context):
        ...

    def health_check(self):
        ...
```

79. `scan()` Contract

Input:

```
target
context
```

Context berisi:

```
config
database
request_manager
api_manager
logger
cache
i18n
```

Output:

```
ModuleResult
```

80. ModuleResult

Concept:

```
ModuleResult(
    module="username",
    target="example",
    status="COMPLETE",
    findings=[...],
    errors=[],
    metadata={}
)
```

81. Finding Contract

```
Finding(
    id=None,
    target="example",
    module="username",
    source="github",
    status="FOUND",
    confidence="HIGH",
    evidence="Public profile verified",
    metadata={
        "profile_url": "...",
        "display_name": "...",
    },
    timestamp="..."
)
```

82. Error Contract

Module error:

```
ModuleError(  
  module="github",  
  reason="Request blocked",  
  status="BLOCKED"  
)
```

Tidak boleh membuat seluruh scan crash hanya karena satu platform gagal.

83. Partial Failure

Misalnya:

```
50 platform
```

dan:

```
40 success  
5 not found  
3 blocked  
1 timeout  
1 error
```

Scan tetap:

```
COMPLETE_WITH_ERRORS
```

bukan:

```
FAILED
```

Karena sebagian besar berhasil.

84. Scan Status

Gunakan:

```
PENDING  
RUNNING  
COMPLETE  
COMPLETE_WITH_ERRORS  
FAILED  
CANCELLED
```

85. API Provider Contract

```
class APIProvider:
    name
    version
    capabilities

    def configure(self):
        ...

    def test(self):
        ...

    def call(self, operation, **kwargs):
        ...

    def health_check(self):
        ...
```

86. Notification Event Contract

```
NotificationEvent(
    type="SCAN_COMPLETED",
    title="Scan completed",
    summary="...",
    data={},
    priority="normal"
)
```

Types:

```
SCAN_STARTED
SCAN_COMPLETED
SCAN_FAILED
HIGH_RISK_FINDING
MONITOR_CHANGE
API_ERROR
UPDATE_AVAILABLE
SYSTEM_ERROR
```

87. Report Contract

```
Report(
    id=...,
    target=...,
    scan_type=...,
    started_at=...,
    completed_at=...,
    summary=...,
    findings=[],
    sources=[],
    metadata={}
)
```

88. Updater Contract

```
class Updater:

    def check(self):
        ...

    def get_latest(self):
        ...

    def verify(self, artifact):
        ...

    def backup(self):
        ...

    def install(self):
        ...

    def rollback(self):
        ...
```

89. i18n Contract

Semua user-facing text:

```
t("key")
```

Bukan:

```
print("Scan completed")
```

langsung.

Contoh:

```
print(t("scan.complete"))
```

90. Apa yang Tidak Boleh Dilakukan Module

Module tidak boleh:

```
menyimpan API key sendiri
membuat database sendiri
membuat HTTP client sendiri
membuat Telegram request sendiri
mengubah locale global
menghapus database
mengubah config tanpa izin
```

Semua melalui core.

91. Dependency Direction

Ini penting supaya tidak terjadi circular import.

Gunakan:

```
CLI
↓
Engine
↓
Modules
↓
Core Services
↓
Infrastructure
```

Bukan:

```
Module A → Module B → CLI → Database → Module A
```

92. Dependency Rule

Idealnya:

```
modules
↓
core interfaces
```

bukan:

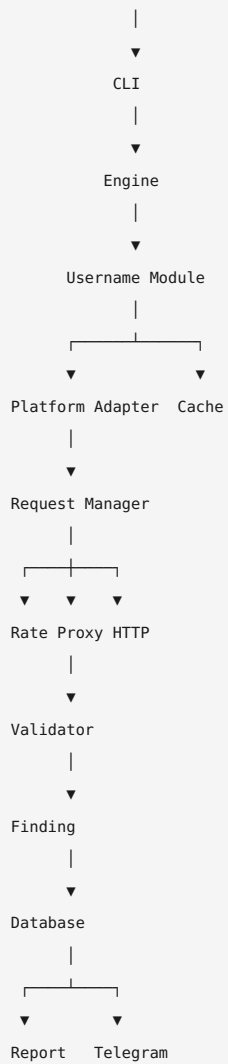
```
core
↓
specific module
```

Core jangan bergantung pada Instagram/GitHub/TikTok secara langsung.

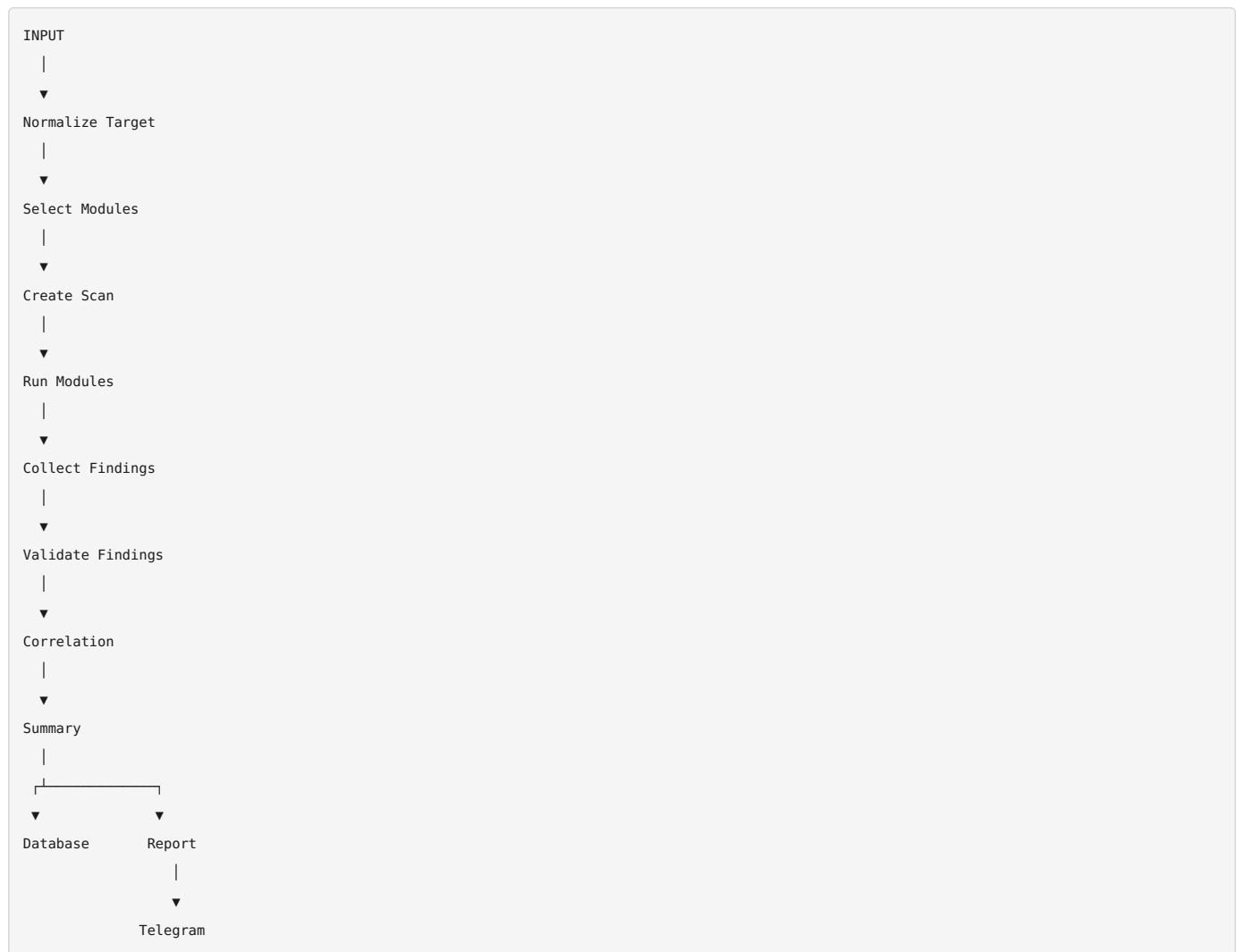
93. Data Flow Final

Contoh username:

```
ghostintel username example --deep
```



94. Contoh Full Scan Flow



95. Test Architecture

```
tests/
├─ unit/
│   ├── test_config.py
│   ├── test_database.py
│   ├── test_username.py
│   ├── test_email.py
│   ├── test_domain.py
│   ├── test_dorking.py
│   ├── test_i18n.py
│   ├── test_api_manager.py
│   ├── test_rate_limiter.py
│   └─ test_reports.py
│
├─ integration/
│   ├── test_scan_engine.py
│   ├── test_api_integration.py
│   ├── test_telegram.py
│   └─ test_database_flow.py
│
├─ e2e/
│   ├── test_username_cli.py
│   ├── test_domain_cli.py
│   └─ test_update.py
│
└─ fixtures/
    ├── username/
    ├── email/
    ├── domain/
    └─ http/
```

96. Testing Principle

Jangan bergantung penuh pada live website.

Mayoritas test:

```
mock
fixtures
fake responses
```

Live test hanya untuk integration tertentu.

Karena website bisa berubah kapan saja.

97. CI

Minimal:

```
Linux
Windows
Python supported versions
```

Termux tidak harus menjadi GitHub Actions runner utama, tetapi harus memiliki manual compatibility test.

98. Security Boundary

```
User Input
↓
Validation
↓
Module
↓
External Source
↓
Sanitization
↓
Finding
```

Jangan percaya:

```
external HTML
external JSON
API response
```

secara mentah.

99. Config Example

examples/config.example.yaml :

```
language: id

database:
  path: "~/ghostintel/ghostintel.db"

network:
  timeout: 15
  retries: 2

rate_limit:
  enabled: true

proxy:
  enabled: false
  url: null

reports:
  default_format: html

telegram:
  enabled: false
  chat_id: null

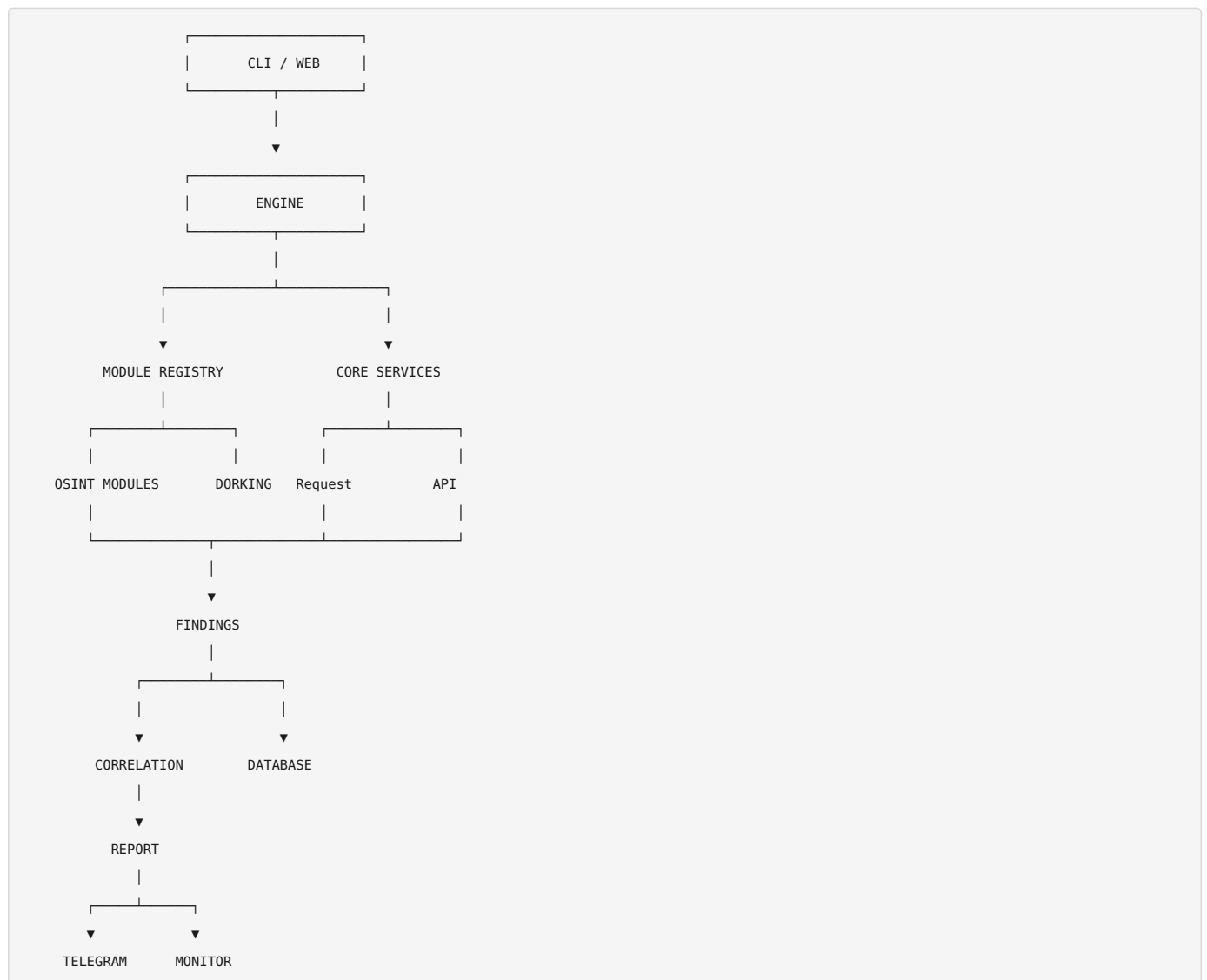
debug:
  enabled: false

updater:
  auto_check: true
  auto_install: false
```

API secrets jangan ditulis di example config.

100. Final Contract

Jadi secara arsitektur:



Dan contract paling pentingnya:

```
CLI/Web
↓
Engine
↓
Module Contract
↓
Core Services
↓
Finding Contract
↓
Database / Report / Notification
```

Dengan struktur ini, ketika nanti kita menambahkan misalnya `Discord Username`, `New Email Provider`, atau `New Reputation API`, kita **tidak perlu bongkar core GhostIntel**. Kita cukup membuat adapter/provider/module yang mengikuti contract yang sudah ditentukan.

Langkah teknis setelah specification ini: implementasi sebaiknya dimulai dari **Phase 1:** `core/models.py` → `exceptions.py` → `config.py` → `database.py` → `logger.py` → `i18n.py` → `request_manager.py` → `api_manager.py` → `module_registry.py`, baru kemudian module OSINT dipindahkan satu per satu. Dengan urutan ini kita menghindari situasi module sudah banyak tetapi fondasinya masih saling bergantung dan akhirnya harus di-refactor ulang.

